

ITI — Technical Instructor (Post-Graduate Training)

Technical Assessment: "Domain Copilot" — An Agentic RAG Platform

Effort: ~40 focused hours · **Window:** 12 calendar days · **AI tools:** allowed and expected · **Stack:** your choice (.NET, Python, Java, Node, Go, ...) · **Submission:** public GitHub repository

1. What You Will Build

An organisation holds a large body of specialised documents. Its experts waste hours searching, cross-referencing and drafting. Build them a **Copilot**: it ingests the document corpus, answers questions **with verifiable citations**, and executes a **multi-step professional workflow** through a team of specialised AI agents — while a human expert approves anything consequential.

Three binding principles:

1. **Grounded, never guessing.** Every claim traces to a source chunk. "Not enough information in the corpus" is a correct and required answer.
2. **The human holds the pen.** At least one workflow step requires explicit human approval.
3. **Everything is observable.** Any agent run can be inspected afterwards: which agent ran, which tools, which chunks, what it cost.

You are **not** expected to finish everything. You are expected to scope an MVP intelligently, build it to enterprise standards, document the gap between what you *would* build and what you *did*, and **be able to teach it**.

This is an instructor role. A repository you cannot explain, defend and turn into a teachable lab does not meet the bar, however good the code. Treat the teaching deliverables as first-class from day one, not as paperwork at the end.

2. Your Assigned Variant

YOUR TASK = Domain Pack $D_{<n>}$ + Mandatory Twist $T_{<n>}$ + the core requirements in §3-§6

Your invitation email states both. If not, derive from your National ID and state the derivation in your README:

- **Domain** = (last two digits) **mod 7** · **Twist** = (sum of all digits) **mod 8**

Variants may not be changed. Building a variant other than the one assigned to you invalidates the submission.

Domain Packs

ID	Domain	Workflow	Required agents (+ orchestrator)	Human approval gate	Risk you must guard
D0	Healthcare — clinical evidence & documentation	Case summary → retrieve guidance → check interactions/contraindications → draft clinical note	Guideline Researcher · Safety Checker · Documentation Drafter	Clinician approves the note	Confident hallucination of dosage or contraindication — must refuse, not infer
D1	Legal — contract review & research	Upload contract → segment clauses → compare to playbook → flag deviations → draft redline + risk memo	Clause Extractor · Risk Assessor · Memo Drafter	Counsel approves the memo	Silent omission — missing a dangerous clause is worse than a false positive
D2	Insurance — claims adjudication	Receive claim → match policy version → evaluate coverage, limits & exclusions → detect anomalies → recommend	Coverage Matcher · Exclusion Analyst · Adjudication Drafter	Adjuster approves the decision	Wrong policy version (retrieval must be version/date aware) and arithmetic hallucination — all limit, deductible and payout calculations run in deterministic code, never in the LLM
D3	Education — curriculum & assessment design	Target role → competency gaps → module outline → generate assessment items + key	Standards Mapper · Curriculum Designer · Item Generator	Lead instructor approves items	Plausible-but-wrong items — automated validation pass required
D4	Government — citizen services & regulations	Citizen describes situation → identify service & eligibility → documents, fees, timelines → draft official response	Eligibility Identifier · Procedure Resolver · Response Drafter	Officer approves the response	Stating an obligation or entitlement that doesn't exist — escalate under ambiguity
D5	Industrial — field maintenance	Symptom → identify equipment & manual revision → diagnostic sequence → safety prerequisites → work order	Symptom Matcher · Diagnostic & Safety Planner · Work Order Generator	Supervisor approves dispatch	Skipping a safety prerequisite — must be structurally enforced, not left to the model
D6	HR — talent screening	Role + candidate pool → extract competency evidence → score against rubric → shortlist + interview probes	Evidence Extractor · Rubric Scorer · Shortlist Drafter	Hiring manager approves shortlist	Bias — protected attributes excluded from scoring, with an audit trail proving it

Corpus: ≥30 documents / 150+ pages, public sources or **synthetic data you generate**. **Never use real personal data — this invalidates the submission outright.**

Mandatory Twists

ID	Twist	Must demonstrably do
T0	Multi-tenancy	≥2 tenants with fully isolated corpora, users and runs; isolation enforced at the data layer; a test proving cross-tenant leakage is impossible
T1	Bilingual (AR + EN)	Arabic documents ingested and retrieved correctly, cross-lingual queries, RTL rendering, Arabic retrieval quality measured separately
T2	Offline / degraded mode	Runs with no internet on a local model; switches automatically on failure; capability differences documented honestly
T3	Cost governor	Per-user token budgets, budget-aware model routing (cheap model for easy steps), pre-flight estimation, hard cut-off, spend view
T4	Audit & replay	Runs persisted in enough detail to replay deterministically; a <code>replay <run-id></code> command; tamper-evident log is a plus
T5	Human review queue	The approval gate becomes a full product: assignment, priority, SLA timers, escalation, approve/reject/edit-with-comment, reviewer stats
T6	Document in / out	OCR of scanned PDFs with confidence handling, plus a professionally formatted generated DOCX/PDF with citations and tables
T7	Async long-running jobs	Real queue and workers; submission returns immediately; progress pushed to client; jobs survive restart, resumable, cancellable, idempotent

3. Core Functional Requirements

- FR-1 Ingestion.** ≥ 2 input formats. Separable, testable stages: extract $\rightarrow$ clean $\rightarrow$ chunk $\rightarrow$ embed $\rightarrow$ index, with document and chunk metadata (source, section, page/clause, version). Idempotent re-ingestion. Per-document status and failure reporting.
- FR-2 Retrieval.** Chunking strategy must be a **deliberate, documented decision** justified against your document structure. **Hybrid retrieval** (dense + keyword) with a documented fusion method. One justified enhancement: re-ranking, query rewriting, HyDE, multi-query, metadata filtering or contextual retrieval. **Citations mandatory** — structured, traceable to the exact chunk. Correct refusal on low-evidence questions.
- FR-3 Evaluation.** A **golden set of ≥ 25 Q/A pairs** including **≥ 5 adversarial cases** (out-of-corpus, ambiguous, prompt injection, conflicting sources), and a **runnable harness** reporting retrieval hit-rate, groundedness and refusal correctness. Record your **actual baseline numbers, including the bad ones**, with interpretation.
- This requirement consistently separates people who have shipped RAG from people who have demoed it. Do not skip it.
- FR-4 Multi-agent.** ≥ 3 specialised agents + orchestrator, each with an explicit role, a **restricted tool set**, defined I/O and a termination condition. ≥ 4 tools, of which ≥ 1 is **write/side-effecting** — never executed without passing the approval gate. Agents communicate through **typed contracts**, not free-form text.
- FR-5 Orchestration.** A named, justified pattern (supervisor, planner-executor, pipeline, state machine). Mandatory controls: **max-iteration breaker, per-step timeout, retry with backoff, graceful degradation** to plain RAG. Every run inspectable step-by-step by run ID. Approval gate supports approve / reject / edit-and-approve, all audited.
- FR-6 Real-time.** Token-level streaming (SSE or WebSocket), **live agent progress events** (not a frozen spinner), and client cancellation that actually stops server-side work.
- FR-7 Surface.** Documented HTTP API (OpenAPI). A **minimal working UI** — web or CLI/TUI — covering ingest, ask with citations, run the workflow, act on the approval gate, view a trace. **Plain and functional is what we want; spend no time on visual design.** Persistent session history.
- FR-8 Access.** Authentication plus ≥ 2 roles with genuinely different permissions, **enforced server-side**, not by hiding buttons.
- FR-9 Observability.** Correlation ID flowing request $\rightarrow$ orchestrator $\rightarrow$ agent $\rightarrow$ LLM call. **Per-request token and cost accounting**, persisted and queryable. LLM tracing via a self-hosted tool, OpenTelemetry, or a clean custom trace store — all equally acceptable. Health/readiness endpoints.

4. Architecture & Engineering

- Clean Architecture is mandatory.** Clean, Hexagonal, Onion or Vertical Slice — name and justify your choice. **Non-negotiable:** your domain and application layers must not depend on any LLM SDK, vector-store SDK or web framework.
- The acceptance test:** swapping the LLM provider, embedding model or vector store must require configuration plus one adapter — not changes to business logic. Make this easy to verify and call it out in your README.
- Provider abstraction (mandatory).** One interface covering completion, streaming, tool calling and embeddings, with **≥ 2 working implementations** (a hosted API and an alternative or local model), selected by configuration, with a documented fallback chain. This is also why you never need to pay for anything: design for a free tier running out.
- Also required:** dependency injection throughout; externalised configuration with **prompts as versioned artifacts**, not string literals; explicitly modelled domain errors; a relational store + a vector store with **migrations**; **≥ 4 ADRs** covering chunking/retrieval, orchestration pattern, vector store choice, and your twist's central decision.
- Testing.** Unit tests for domain and application logic with LLM calls stubbed; integration tests for ingestion and retrieval; contract tests for tool and agent schemas. We read your tests; we do not count them. Ten sharp tests beat a hundred trivial ones.
- Packaging.** `docker compose up` (or one equivalent documented command) brings up the whole system including databases, plus a seed/ingest command. `.env.example` complete, with no real secrets.

5. Security

- Document every control against the threat it addresses in `docs/SECURITY.md`.
- OWASP Web Top 10** — broken access control (object-ownership checks), cryptographic failures, injection (parameterised queries, validated uploads), rate limiting and abuse controls, security misconfiguration (headers, CORS, no defaults), dependency scanning in CI, and auditable security logging that never records secrets.
- OWASP LLM Top 10** — - **Prompt injection**, direct and **indirect via ingested documents**: sanitisation, strict privilege separation between instructions and retrieved content, and **≥ 3 injection cases in your evaluation set that your system demonstrably resists**. The indirect case is the one that matters most here. - **Insecure output handling**: never render model output as raw HTML or pass it to a shell, SQL or file path unvalidated; tool arguments schema-validated before execution. - **Sensitive disclosure**: PII detection/redaction; document what leaves your infrastructure. - **Excessive agency**: per-agent tool allow-lists; no destructive tool without the approval gate. - **Unbounded consumption**: token caps, iteration limits, rate limits, payload limits. - **Supply chain**: pinned dependencies, committed lockfiles, scanning in CI.
- Secrets:** none in the repository, ever — including in history. Run a secret scanner over your full history before submitting. A leaked key is a serious finding, because you would be teaching that habit.

6. Engineering Process

- We read your repository as a story, not a snapshot.** Excellent code delivered as a single `initial commit` does not meet this section's requirements.
- ≥ 30 meaningful commits across ≥ 6 distinct days.** Distribution is checked. Conventional Commits or another consistent, documented convention. Messages explain **why**. Atomic commits — no 4,000-line dumps, no `WIP / fix2 / final-final`, no commented-out code. Correct `.gitignore` from commit one.
 - All work merges via Pull Request** — no direct pushes to `main`, even solo; this is the discipline you will be teaching. **≥ 8 PRs** with real descriptions (what, why, how tested), each **self-reviewed with inline comments**.
 - GitHub Issues** linked to PRs (`Closes #12`), plus a board or milestones showing your plan and what you deliberately deferred.
 - GitHub Actions CI** on every PR: build, lint, tests, dependency and secret scanning. **Green on main at submission**. Branch protection enabled. Release tags marking increments.
 - Repo hygiene: `README`, `LICENSE`, `CONTRIBUTING.md`, `.env.example`, PR/issue templates, `CODEOWNERS`.
- Agentic coding workflow (mandatory).** Because AI use is allowed, *how deliberately you direct it* is part of what you submit. Commit your agentic configuration and explain it in `docs/AGENTIC-WORKFLOW.md`. Include **at least five** of: a project instruction file encoding your architecture rules so the AI respects your boundaries; reusable versioned prompt assets or skills; sub-agents scoped to distinct roles (security reviewer, test writer, documentation); hooks enforcing quality gates automatically; custom commands for repeated operations; MCP server usage or a custom MCP server; a versioned prompt library for the product. Explain what you configured, **why**, what it changed, and **where the agentic approach failed you**.

You are not "using AI to write code." You are engineering a development system in which an AI is a governed participant. That distinction is what you would be teaching.

Maintain `docs/AI-USAGE-LOG.md`: what you delegated, what you wrote yourself, where the AI misled you, how you verified. **Be honest about AI mistakes** — catching model errors is exactly the skill we need you to teach, and a log claiming flawless usage tells us nothing.

7. Deliverables

1. **Public GitHub repository** — all source, agentic config, docs, migrations, seed data, Docker setup. Public for 30 days.
2. **BRD** (`docs/BRD.md`) — context, personas, objectives with measurable criteria, uniquely-ID'd requirements with acceptance criteria, **explicit out-of-scope**, business rules, assumptions, risks, and a **traceability matrix** (BR-xx → implemented / partial / deferred → evidence).
3. **System Design Document** (`docs/SYSTEM-DESIGN.md`) — the most important document, in **two separated parts**: - **Part A — Target architecture**, unconstrained: gateway and managed rate limiting, secrets manager, broker, autoscaling, caching, managed vector DB, observability stack, CI/CD environments, DR/backup, cost model at scale. - **Part B — Implemented MVP**, with a **gap table**: *target component | implemented? | why deferred | interim mitigation | effort and cost to close*. - Plus **every significant design decision** with alternatives considered and why they were rejected. Where you made an expedient or poor choice under time pressure, say so — candour here works in your favour.

Part B's reasoning matters more than Part A's size. "Not implemented, needs a paid tier" is weak. "Deferred managed rate limiting; it mitigates my top operational risk (abuse-driven token spend), so I built an in-process token bucket, accepting it won't survive horizontal scaling; closing the gap ≈ 4h + \$X/month" is the answer of someone ready to teach architecture.

4. **Architecture docs** (`docs/ARCHITECTURE.md`) — C4 Levels 1–3, a sequence diagram of the full agentic workflow including the approval gate and streaming, a data-flow diagram marking trust boundaries and **what the LLM provider sees**, an ER diagram, a layer-dependency diagram, and your ≥4 ADRs. **Diagram source committed**, not only images.
5. **README** — assume the reader has Docker, nothing else, and 15 minutes: prerequisites, quick start, every environment variable, how to obtain free API keys, **how to run with a local model if no key is available**, how to run tests and the evaluation harness, seeded demo accounts, troubleshooting, and a **"5-Minute Demo Path"** — a numbered script showing every core capability. Write this; it is how your work gets seen.
6. **Security & evaluation reports** — `docs/SECURITY.md` and `docs/EVALUATION.md` with real numbers and failure analysis.
7. **Teaching pack** (`teaching/`) — built from your own system: 15–25 slides for a 90-minute post-graduate session on one topic from your build (RAG beyond the demo / multi-agent orchestration / OWASP LLM in practice); a **hands-on lab sheet** a trainee performs against your repository, with expected outputs, ≥3 stretch challenges and an answer key; a learning-outcomes and assessment map; and a one-page **"common trainee mistakes"** note — the 5 misconceptions you predict and how you'd correct each.
8. **Two videos** (unlisted links in README) — a **5–8 min product demo** (ingest, grounded answer with citations, correct refusal, multi-agent run with live progress, approval gate, trace, your twist) and a **10 min teaching sample** delivering a slice of your session as if to trainees, **face and voice required**. Do not read slides aloud — teach.
9. **Deployment (optional, strongly valued)** — any free tier. Live URL and demo credentials. IaC or a documented CD pipeline counts for more than a manual deployment.

8. Scoping — How to Spend 40 Hours

Depth beats breadth. Every requirement is worth more delivered *well and partially* than *fully and badly*.

If you fall behind, cut in this order: UI richness → number of agents (3 + orchestrator is the floor) → the optional retrieval enhancement → corpus size (30 documents is the floor) → breadth of your twist (implement its core, document the rest).

Never cut: the evaluation harness, security controls, documentation, the teaching pack, or the videos. **Document every cut in the SDD gap table** — a well-documented cut costs you far less than a silent one.

9. Rules & Submission

Individual work — sharing code, docs or prompts with another candidate invalidates the submissions of **all** parties. Any stack; what matters is how you use it, not which one you pick. **No paid subscriptions required** — free API tiers or a local model are fully acceptable, which is why provider abstraction is mandatory. Synthetic or public data only. Declare any template or starter repo you build on, in your README and first commit. You must be able to explain and justify any line of what you submit; claiming an unimplemented feature is worse than omitting it. Reasonable ambiguity in this brief is intentional — resolve it and record your assumptions in the BRD.

Non-negotiables: no real personal data in the repository · no plagiarised or unattributed existing solution · no secrets in Git history · no documentation claiming functionality that does not exist · CI green on `main` · teaching pack and both videos present.

Deadline: 23:59 Cairo time on day 12 from your invitation email; the latest commit on `main` is your submission time.

Submit by replying to your invitation email, subject `ITI Instructor Task - <Full Name> - D<n>T<n>`, with: repository URL · demo video · teaching video · deployment URL and credentials (if any) · your declared variant and its derivation · actual hours spent (honest — it calibrates our reading, nothing more) · one paragraph on what you're proudest of and what you'd fix first with another 40 hours.

Before you submit: clone fresh and verify `docker compose up` works on a clean machine — do not assume. CI green, branch protection on. Secret scan run over full history. ≥30 commits / ≥6 days / ≥8 PRs. All documents present. Both video links tested in a private browser window.

Questions: reply to your invitation email. Good luck — we look forward to reading your work.